Задания №19-21, программирование | Часть 1

Уровень: "Нет данных"

Задание №1

Полина и Варя играют в одну игру. Смысл игры заключается в следующем:
1) Перед игроками лежит одна куча камней. Игроки ходят по очереди, первый ход делает Полина.
2) За один ход игрок может добавить в кучу 1 камень или увеличить количество камней в куче в 3 раза. Для этого у каждого игрока есть неограниченный запас камней.
3) Игра завершается, когда количество камней в куче стало не менее 56.
4) Победителем является тот игрок, который сделал последний ход и получил сумму камней в куче, равную 56 или больше.
Пример: имея кучу из 15 камней, за один ход игрок может получить кучу из 16 или 45 камней.
В начальный момент в куче было S камней в диапазоне [1, 55].
Примечание. Игрок обладает выигрышной стратегией, если у него есть возможность набрать нужную сумму камней в куче для выигрыша при любых ходах противника.
Мы знаем, что Варя выиграла своим первым ходом после неудачного хода Полины. Определите минимальное начальное количество камней в куче, при котором возможна такая ситуация.

Верные варианты

7

Решение

INSANEK.RU

Сообщить об ошибке

Решение программированием. Способ 1:
1. Объявляем функцию g(s, p, end), которая будет вычислять результат игры, где s — текущее количество камней в куче, p — текущий ход игры, end — количество ходов, за которое должна завершиться игра.
2. Внутри функции пропишем, что если количество камней в куче не менее 56 и игразавершилась за необходимое количество ходов, то вернем истину, в остальных случаях —ложь.
3. Если следующий ход будет такой же четности, что и последний, то нам нужна хотя бы одним ходом, иначе — всеми вариантами ходов.
4. Запустим цикл, в котором будет проверяться, что при текущем значении s игра завершается победой Вани при неудачном ходе Пети.
Код:

```
def g(s, p, end):
    if s >= 56 and p in end:
        return True
    if s < 56 and p == max(end):
        return False
    if s >= 56 and p not in end:
        return False
    if (p + 1) % 2 == (end[0] % 2):
        return g(s + 1, p + 1, end) or g(s * 3, p + 1, end)
    else:
        return g(s + 1, p + 1, end) or g(s * 3, p + 1, end)

for s in range(1, 56):
    if g(s, 0, [2]):
        print(s)
        break
```

Решение руками:
Имеются команды: +1, *3. Победа, если в куче 56 камней и более.
Так как по условию требуется неудачный ход, быстрее всего можно получить выигрыш дважды использовав сильный ход, то есть *3.
Тогда, S * 3 * 3 >= 56, S >= 6.2.
Ответ: 7.
Решение программированием. Способ 2:
1. Создаём функцию, которая будет проверять, является ли текущая позиция выигрышной за один ход. Назовем её Win1, а в качестве параметра будем передавать возможное начальное значение S.
Функция будет возвращать результат выражения ((S + 1 >= 56) or (S * 3 >= 56)). То есть результатом будет True (если позиция подходит) или False (если позиция не подходит). Это помогает узнать, можно ли за один ход получить победное (>=56) количество камней в куче:

```
def Win1(S):
    return ((S + 1 >= 56) or (S * 3 >= 56))
```

2. Создаём список, в котором будут храниться все подходящие под условие значения S:

```
S19 = []
```

3. Создаём цикл для перебора возможных значений S. Диапазон перебора указан в условии задачи:

```
for S in range(1, 56):
```

4. Делаем проверку:

```
if Win1(S + 1) or Win1(S * 3):
```

Она работает так: если после любого из ходов Полины (нас устроит хотя бы один подходящий вариант, поэтому пишем or) можно будет победить за 1 ход, то такая позиция подходит и мы добавим её в список:

S19.append(S)
5. Выводим на экран минимальное значение из списка — это и будет ответом:
print(min(S19))
Полный код:
```
def Win1(S):
   return ((S + 1 >= 56) or (S * 3 >= 56))
S19 = []
for S in range(1, 56):
   if Win1(S + 1) or Win1(S * 3):
      S19.append(S)
print(min(S19))
```

Задание №2

Полина и Варя играют в одну игру. Смысл игры заключается в следующем:
1) Перед игроками лежит одна куча камней. Игроки ходят по очереди, первый ход делает Полина.
2) За один ход игрок может добавить в кучу 1 камень или увеличить количество камней в куче в 3 раза. Для этого у каждого игрока есть неограниченный запас камней.
3) Игра завершается, когда количество камней в куче стало не менее 56.
4) Победителем является тот игрок, который сделал последний ход и получил сумму камней в куче, равную 56 или больше.
Пример: имея кучу из 15 камней, за один ход игрок может получить кучу из 16 или 45 камней.
В начальный момент в куче было S камней в диапазоне [1, 55].
Примечание. Игрок обладает выигрышной стратегией, если у него есть возможность набрать нужную сумму камней в куче для выигрыша при любых ходах противника.
INSANEK.RU
Найдите минимальное значение S, при котором параллельно выполняются два условия:
— у Вари есть стратегия, позволяющая ей выиграть первым или вторым ходом при любой игре Полины;
— у Вари нет стратегии, которая позволит ей гарантированно выиграть первым ходом.

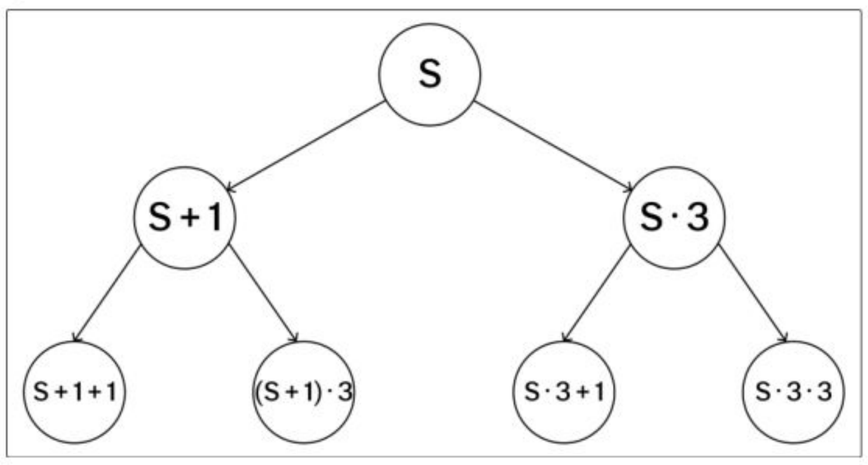
Верные варианты

16

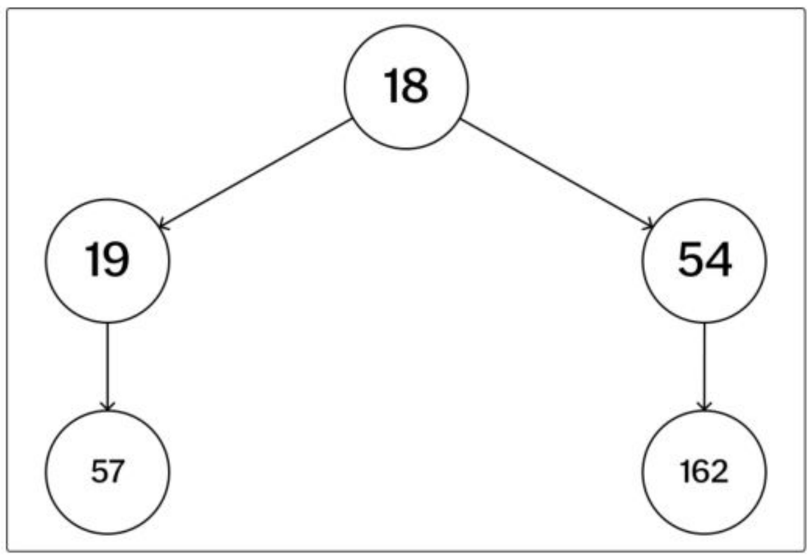
Решение

Сообщить об ошибке

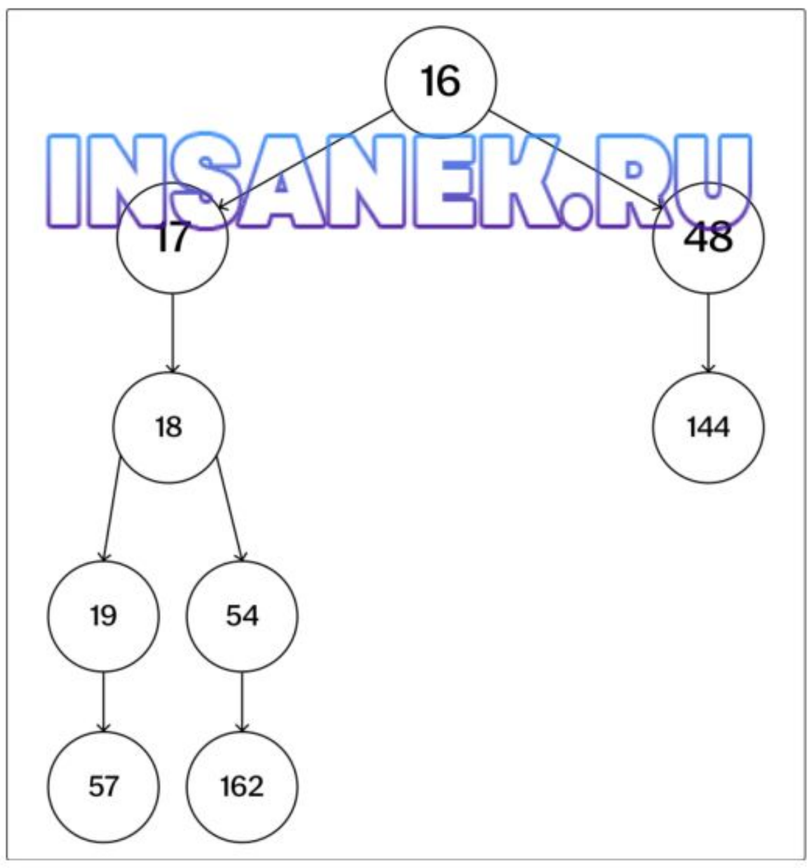
Решение программированием. Способ 1:
1. Объявляем функцию g(s, p, end), которая будет вычислять результат игры, где s — текущее количество камней в куче, p — текущий ход игры, end — количество ходов, за которое должна завершиться игра.
2. Внутри функции пропишем, что если количество камней в куче не менее 56 и игразавершилась за необходимое количество ходов, то вернем истину, в остальных случаях —ложь.
3. Если следующий ход будет такой же четности, что и последний, то нам нужна хотя бы одним ходом, иначе — всеми вариантами ходов.
4. Запустим цикл перебора всех значений S, внутри цикла будем искать все значения, при которых игра может завершиться за 2 или за 4 хода.
Код:
```
def g(s, p, end):
   if s >= 56 and p in end:
      return True
   if s < 56 and p == max(end):
      return False
   if s >= 56 and p not in end:
      return False
   if (p + 1) % 2 == (end[0] % 2):
      return g(s + 1, p + 1, end) or g(s * 3, p + 1, end)
   else:
      return g(s + 1, p + 1, end) and g(s * 3, p + 1, end)
for s in range(1, 56):
   if g(s, 0, [2, 4]) and not g(s, 0, [2]) :
      print(s)
```
Решение руками:
Имеются команды: +1, *3. Победа, если в куче 56 камней и более.



Наименьшее значение S, из которого можно выиграть за 1 ход – 19 (при умножении на 3 получается значение, большее 55).
Далее нам нужно найти проигрышную позицию. Проигрышная позиция — позиция, совершая ход из которой при любом ходе попадет в выигрышную позицию, тем самым соперник сможет выиграть. Позиция 18 в таком случае является проигрышной, так как после любого хода из нее соперник будет ходить из значения, большего или равного 19, откуда он сможет выиграть за 1 ход.



Полина сможет гарантированно выиграть за 2 хода, если передаст позицию 18 Веронике. Сделать это возможно из позиции 17 ходом +1 и из позиции 6 ходом *3.
Необходимо найти такую позицию, сделав ход из которой, Полина одним своим возможным ходом создаст позицию, выигрышную за 1 ход, а другим – выигрышную за 2 хода. Мы уже знаем, что 6 и 17 – позиции, выигрышные за 2 хода. В позицию 17 можно попасть из 16 ходом +1. Проверим, чтобы второй возможный ход Полины создавал позицию, проигрышную за 2 хода:



Позиция 16 подходит.
В 7 можно попасть из 6 ходом +1. Но второй возможный ход из позиции 7 слишком слабый, чтобы передать победу за 1 ход. В таком случае 16 – ответ на вопрос задачи.
<u>Решение программированием:</u>
1. Создаём функцию, которая будет проверять, является ли текущая позиция выигрышной за один ход. Назовем её Win1, а в качестве параметра будем передавать возможное начальное значение S.
Функция будет возвращать результат выражения ((S + 1 >= 56) or (S * 3 >= 56)). То есть результатом будет True (если позиция подходит) или False (если позиция не подходит). Это помогает узнать, можно ли за один ход получить победное (>=56) количество камней в куче:
```
def Win1(S):
   return ((S + 1 >= 56) or (S * 3 >= 56))
```
2. Создаём функцию, которая будет проверять, является ли текущая позиция проигрышной за 1 ход. Проигрышная за 1 ход позиция — это позиция, из которой соперник 100% выиграет, как бы мы ни сходили (причём выиграет своим первым ходом).
Функция будет возвращать результат выражения ((not(Win1(S))) and Win1(S + 1) and Win1(S * 3)). Что должно выполняться, чтобы функция вернула True?
1) Текущая позиция не должна быть выигрышной за 1 ход (not(Win1(S))).
2) Как бы ни сходил игрок, у него должна получаться такая позиция, из которой соперник победит своим первым ходом.
```
def Loss1(S):
   return ((not(Win1(S))) and Win1(S + 1) and Win1(S * 3))
```
3. Создаём функцию, которая будет проверять, является ли текущая позиция выигрышной за два хода. То есть может ли игрок, который будет из неё ходить, победить своим вторым ходом.
Функция будет возвращать результат выражения (Loss1(S + 1) or Loss1(S * 3)). Что должно выполняться, чтобы функция вернула True?
У игрока, который делает ход из этой позиции, должен быть хотя бы один вариант, который сможет принести проигрыш сопернику. То есть один из возможных ходов должен создать проигрышную за 1 ход позицию (поэтому пишем or), чтобы соперник не смог победить, а сам игрок своим вторым ходом одержал победу.
```
def Win2(S):
   return (Loss1(S + 1) or Loss1(S * 3))
```
4. Создаём функцию, которая будет проверять, является ли текущая позиция проигрышной за 1 или 2 хода (Варя выигрывает за 1 или 2 хода, следовательно, Полина проигрывает за 1 или 2 хода).
Функция будет возвращать True или False в зависимости от значения выражения. Истина возможна при выполнении условия: позиции, которые можно получить из текущей, 100% выигрышные — либо за 1 ход, либо за 2.
Чтобы исключить вариант, что все позиции, которые можно получить из S, выигрышные только за 1 ход, добавим условие (Win2(S + 1) or Win2(S * 3)). Оно обеспечит хотя бы один вариант победы за 2 хода.

def Loss12(S):
return ((Win1(S + 1) or Win2(S + 1)) and (Win1(S * 3) or Win2(S * 3)) and (Win2(S + 1) or Win2(S * 3)))
5. Создаём список, в котором будут храниться все подходящие под условие значения S:
S21 = []
6. Создаём цикл для перебора возможных значений S. Диапазон перебора указан в условии задачи:
for S in range(1, 56):
7. Делаем проверку:
if Loss12(S):
Она работает так: если из текущей позиции можно проиграть первым или вторым ходом, то такая позиция подходит и мы добавим её в список:
S21.append(S)
7. Выводим на экран минимальное значение списка:
print(min(S21))
Полный код:

```
def Win1(S):
   return ((S + 1 >= 56) or (S * 3 >= 56))
def Loss1(S):
   return ((not(Win1(S))) and Win1(S + 1) and Win1(S * 3))
def Win2(S):
   return (Loss1(S + 1) or Loss1(S * 3))
def Loss12(S):
   return ((Win1(S + 1) or Win2(S + 1)) and (Win1(S * 3) or Win2(S * 3)) and (Win2(S + 1) or Win2(S * 3)))
S21 = []
for S in range(1, 56):
   if Loss12(S):
      S21.append(S)
print(min(S21))
```

INSANEK.RU